

- Secrets
- ABAP
- Ansible
- Apex
- AzureResourceManager
- C
- C#
- C++
- CloudFormation
- COBOL
- CSS
- Dart**
- Docker
- Flex
- Go
- HTML
- Java
- JavaScript
- JCL
- Kotlin
- Kubernetes
- Objective C
- PHP
- PL/I
- PL/SQL
- Python
- RPG
- Ruby
- Scala
- Swift
- Terraform
- Text
- TypeScript
- T-SQL
- VB.NET
- VB6
- XML



Dart static code analysis

Unique rules to write Clean DART Code

All rules **115** Bug **15** Code Smell **100**

Tags ▾

Impact ▾

Clean code attribute ▾

Search by name...

Code Smell
Void functions should not return null
Code Smell
Function declarations should be preferred over variables
Code Smell
Extension identifiers should comply with a naming convention
Code Smell
Local identifiers should not start with underscore
Code Smell
If-null operator should be preferred
Code Smell
Null-aware operators should be preferred
Code Smell
Generic function type aliases should be preferred
Code Smell
Type parameters should not shadow other type parameters
Code Smell
"static final" declarations should be "const" instead
Code Smell
Interpolation should be used instead of String concatenation
Code Smell
Collection literals should be preferred
Code Smell

Interpolation should be used instead of String concatenation

Analyze your code

Intentionality - Clear Maintainability

Code Smell Minor es2015 clumsy

Why is this an issue? More Info

In Dart there’s an ability to use template literals instead of concatenation. Since their use is clearer and more concise, they are preferred.

Exceptions

Concatenation of string literals, without any variables, is allowed, both using + and using adjacent strings. Those are typically used for multiline strings.

Raw string literals are also an exception to this rule, since they don’t support interpolation.

```
var s1 =
  'hello\n' +
  'world'; // OK
var s2 =
  'hello\n'
  'world'; // OK
var s3 = r'hello\n' + s1; // OK
```

The multiline strings like s1 and s2 above can also be written as follows:

```
var s1Alternative = '''
hello
world'''; // OK
```

Noncompliant code example

```
void sayHello(name) {
  print('hello ' + name + '!');  // Noncompliant
}
```

Compliant solution

```
void sayHello(name) {
  print('hello $name!');
}
```

Available In:

sonarcloud

